prompting-explained

Please, Thank You, and Other Words the AI Ignored

A teaching tool that shows you the impact of each phrase in your prompt on the AI's answer.

Vihaan Manchanda

Duke University · Explainable AI

TRY IT AT THE POSTER, OR ONLINE

github.com/Vihaan8/prompting-explained

01 Executive summary

Why. Staff using AI tools at work get no feedback on which parts of their prompts the model acted on, so they can't tell good habits from bad ones.

What we built. prompting-explained removes one phrase at a time, reruns the prompt, scores each answer, and shows which phrases changed the result.

What we found. On 240 test prompts across 8 everyday tasks with one model, the phrases it ranked highest mattered more than random ones (score drop 0.30 vs 0.18), and a rerun picked the same top phrase 78% of the time.

Goal. A training tool companies load with their own tasks, so employees learn which prompting habits change results on their real work.

02 The problem

Users can't see how the AI read their prompt. Teachers, marketers, nurses, students and support agents use AI at work, but the tools give no feedback on which words shaped the answer.

Bad habits go unnoticed. Without feedback, people can't tell which of their habits help and which backfire. In our tests, "dont guess" made the AI speculate *more*, in all 5 runs.

Mistakes reach real work. Asked to "turn this ticket into a bug report" for a public tracker, the AI included the customer's name in 2 of 3 runs, and their email in one. Nothing in the answer warned the user.

03 Who it's for

Anyone learning to prompt. Practice on everyday tasks and see how the AI read your words.

Companies. Add your own tasks, documents and rules, so staff learn on the work they actually do.

Governance teams. Test whether a safety instruction works before rolling it out.

RESEARCH QUESTION

Which parts of an everyday prompt actually change an AI's answer, and can removing parts and rerunning explain that **faithfully** and **stably**?

04 Methods

How the tool works out what each phrase did, in four steps. Starred terms are explained in the purple notes.

1 Split the prompt into phrases

An AI splits the prompt into phrases and tags each one (task, audience, format, politeness...). These phrases are the pieces the tool will perturb*. Code keeps each phrase exactly as typed, typos included.

*** Perturbation:** changing part of an input to see how the output responds. LIME does this with single words.

2 Remove one phrase and rerun

The tool deletes one phrase, runs the shorter prompt again, and compares the two answers. This is occlusion*. A phrase's effect is how much the answer's score drops without it, so every phrase gets its own counterfactual**: what if you hadn't written this?

*** Occlusion (leave-one-out):** hide one part of the input, rerun, and measure the change.
**** Counterfactual:** the same case with one thing changed, to see what that one thing did.

3 Score every answer two ways

Against the task. Each task comes with a short checklist that code grades automatically, like a teacher's rubric. Every answer gets a score from 0 (misses everything) to 1 (passes everything).

Against your instructions. Code can't check everything you might ask for, such as "use a candy example". So a second AI reads each answer and ticks off which of your instructions it followed*. That tells "the AI followed this" apart from "this made the answer better".

*** AI as a judge:** using one AI to grade another's output. It can be wrong, so its mistakes are reported.

4 Filter out noise

The same prompt gives a slightly different answer each time, so every version runs more than once. A change only counts if it's bigger than the noise floor*. Anything smaller stays grey, and the tool says "not clear" rather than guessing.

*** Noise floor:** the smallest change the tool can tell apart from the AI's natural randomness (two standard errors).

05 Worked example

Task: write a prompt that gets the AI to explain inflation to your 10-year-old cousin. One user wrote this:

> Hi there! Explain inflation to my 10-year-old cousin. Use exactly 3 bullet points. Mention a candy bar in one of them. Use proper technical terms.

Made the answer better · Followed, no change in score · Worked against the task · No real difference

Green · "Explain inflation": without it, the AI explained how the body uses food for energy.

Blue · "Mention a candy bar": the AI did it, but the score was the same either way.

Red · "Use proper technical terms": the AI used words like "Consumer Price Index", too hard for a 10-year-old. Without it, the score rose from 0.40 to 0.78.

Grey · "Hi there!": a greeting gives the AI nothing to act on.

Scores run 0 to 1. On a fresh rerun, 5 of 6 verdicts repeated. The "without it" sentences are written by an AI and labelled as descriptions; every number is computed by code.

06 What worked

240 test prompts on 8 tasks, written the way novice, average and expert users would, plus 32 deliberately messy prompts. One model.

1 It pointed at the phrases that mattered

Deleting the 2 phrases the tool ranked highest dropped the score by **0.30**, against **0.18** for 2 random phrases (79 prompts, p = 0.0002), so its explanations are faithful*.

2 Its results repeated

A full rerun picked the same top phrase **78%** of the time, and 86% when its effect was clear, so they are stable**.

*** Faithfulness:** the parts an explanation points to really drive the output.
**** Stability:** the same input gets the same explanation when run again.

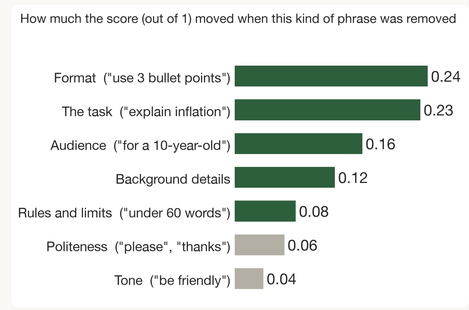
TRY IT ON YOUR PHONE

Scan to open the app

Pick a task, write a prompt, and see which of your words changed the answer. Free, no sign-up.

3 Format and task instructions moved the score most

Across 741 phrases, removing a format instruction or the task itself moved the score most. Removing politeness or tone barely moved it.



4 The AI didn't always take instructions literally

"dont guess" made it speculate *more*, in all 5 runs. Told to "make up a salary", it refused. "for github issues" made it leave a customer's name out of a bug report without being asked.

07 What didn't work

- **Nothing to rank when a document did the work.** On the landlord and meeting tasks, removing any 2 phrases moved the score by under 0.07.
- **The AI judge couldn't count words.** It agreed with code 88% of the time, but passed 93 answers that broke a word limit.
- **More reruns barely helped.** Nearly tripling the calls raised agreement from 83% to 91%.
- **Our hallucination test failed.** The model never invented a salary, even with a bonus in the posting as bait.

08 Business implications

Train employees on good habits. Companies load their own tasks, documents and rules, so staff practice on real work and see which habits change results.

Explainability for non-AI staff. Anyone can see how the model read their words, without an AI background.

Stretch goal: cheaper queries. With enough measured phrases, results can be cached and a small local model trained, so an app or extension filters a prompt before it is sent, with no extra AI call. That could cut token costs.

09 Limitations and next steps

- One model (Claude Haiku 5.5), on tasks with checks written in code.
- Test prompts were written by an AI imitating users, plus 32 by me. No study with real users yet.
- Each phrase is compared only with deleting it, not with a better phrase.
- Phrases that do nothing alone are retested as one group, but not in every combination. Shapley values* would, at a much higher cost.
- The splitter and instruction checker are AIs and can be wrong. Checked against code, the checker agreed 88% of the time but can't count words.
- **Next:** Shapley-style scoring, a study with real learners, more models.

*** Shapley values:** give each part a fair share of the credit by testing it in every combination of parts.

REFERENCES Zeiler & Fergus (2014), occlusion · Ribeiro et al. (2016), LIME · Lundberg & Lee (2017), SHAP · Petsiuk et al. (2018), deletion test · DeYoung et al. (2020), faithfulness · Alvarez-Melis & Jaakkola (2018), stability · Turpin et al. (2023), unfaithful model explanations